



الجامعة الإسلامية - غزة
كلية تكنولوجيا المعلومات
ويب 2 عملي
SDEV 2106, WDMM 2010,
MOBC 2102
الفصل الثاني: 2024/2025
المدرس: محمد زقلام



Assignment - 2 (PHP Arrays and Forms)

Objective:

Build a dynamic, single-file PHP web application that manages a personal book library. Users can browse a list of books, add new ones through a validated form, and receive clear feedback — all styled with Bootstrap 5. The application must also handle data persistence on failed validation and be version-controlled with GitHub.

Part 0: Project Setup & Bootstrap:

- **Bootstrap CDN:** Link Bootstrap 5 CSS and JS via CDN in the <head> of index.php — no local files.
- **Page layout:** Wrap all content in a .container. Use .row and .col-* for the two-column layout (form left, table right on wider screens).
- **PHP at the top:** All PHP logic — including session_start() — must appear before the <!DOCTYPE html> declaration.

Part 1: Data Structure & Initialization (PHP Arrays)

- **\$books array:** Create a multi-dimensional PHP array named \$books. Each element is an associative array representing one book.
- **Required keys:** id, title, author, genre, year, pages. Pre-load at least three sample books.
- **Genre list:** Store allowed genres in a separate \$genres array: ["Fiction", "Non-Fiction", "Science", "History", "Biography", "Technology"]. Use this array for the dropdown — never hardcode <option> tags.
- **ID generation:** When adding a new book, generate its id by finding the maximum existing id and adding 1. Do not use count().

Part 2: Form Handling & Validation Logic (PHP)

- **Detect POST:** Check `$_SERVER["REQUEST_METHOD"] === "POST"` to detect a form submission.
- **\$errors array:** Initialise `$errors = []`. Store field-specific messages like `$errors["title"] = "Title is required."`
- **Sanitize first:** Use `trim()` and `htmlspecialchars()` on every input before validation.
- **title rules:** Required. Length must be between 3 and 120 characters.
- **author rules:** Required. Must contain at least two words (first + last name).
- **genre rules:** Required. Must be a value that exists in the `$genres` array. Validate using `in_array()`.
- **year rules:** Required. Must be a 4-digit integer between 1000 and current year (use `date("Y")`).
- **pages rules:** Required. Must be a positive integer greater than 0.
- **\$submittedData:** Always store sanitized POST values in `$submittedData` so the form can be re-populated on error.
- **On success:** If `$errors` is empty: generate new id, push to `$books`, set `$_SESSION["success"]` message, clear `$submittedData`, redirect with `header()` to prevent re-submission on refresh.

Part 3: HTML Output & Bootstrap Integration

- **Success alert:** Read and unset `$_SESSION["success"]`. Display it in a dismissible alert-success Bootstrap alert.
- **Error alert:** If `$errors` is not empty, show a general alert-danger alert at the top of the form section.
- **Field errors:** Add class `is-invalid` to any input that has an error. Display the message in a `<div class="invalid-feedback">` immediately after the input.
- **Book table:** Use Bootstrap classes: `table table-striped table-hover table-bordered`. Show columns: #, Title, Author, Genre, Year, Pages. Format year as a plain integer.
- **Form fields:** Use `form-label` and `form-control` on every field. Wrap in `mb-3` divs. The genre field must be a `<select>` populated by looping over `$genres`.
- **Re-population:** Use `$submittedData` to re-fill `value=""` attributes and `textarea` content. For the genre `<select>`, add `selected` to the matching `<option>`.
- **Security:** Wrap every echo of user data in `htmlspecialchars()`.

Part 4: GitHub Submission

- **Repository name:** Create a public repository named: php-book-library
- **Single file:** Your entire solution must be in one file: index.php
- **Commit message:** Your initial commit message should describe what the file does, not just say "first commit".
- **Submission:** Submit the GitHub repository URL in a plain .txt file alongside the zipped index.php.

Part 5: Challenge Features (Required & Optional)

The optional features will be graded with bonus marks. Each feature must work correctly and not break the core functionality.

- **Edit a book(required):** Use a ?edit_id= query parameter to detect edit mode. Pre-populate the form and change the button text to "Update Book".
- **Delete a book(required):** Add a Delete button per row. Submit a hidden POST form with the book id. Remove the entry from \$books using array_filter() and re-index with array_values().
- **Search / filter(optional):** Add a search bar above the table. Filter \$books by title or author using a loop and strpos(). Preserve the search term in the input.
- **Sort the table(optional):** Add clickable column headers that sort the table by that column using usort() with a custom comparison function.
- **Bootstrap Modal confirmation(required):** Wrap the delete action in a Bootstrap Modal that asks "Are you sure?" before submitting.
- **Cover image(optional):** Add an image_url field. Display a small thumbnail in the table. Validate that the URL ends with .jpg, .jpeg, .png, or .gif.

Submission Guidelines

File requirements:

- All PHP logic must be in a single file named index.php.
- session_start() and all PHP processing must appear before <!DOCTYPE html>.
- Submit a .zip archive containing only index.php and .txt file
- Submit the GitHub repository URL in a separate plain .txt file.
- The repository must be public and named php-book-library.

Code quality requirements:

- Bootstrap must be loaded from a CDN — no local CSS or JS files.

- All user-echoed data must be wrapped in `htmlspecialchars()`.
- Code must be well-structured with comments explaining each major section.
- The application must run without PHP errors or warnings.
- Validation must cover all 5 fields with specific, field-level error messages.

Cheating between students will result in a mark of zero for both parties. Identical code structures, variable names, and comment wording are all flagged during grading.

Deadline: Friday, 8/5/2026. Late submissions will not be accepted without prior arrangement.

AI Integration Guide — Using AI Responsibly

This section is your practical guide for using AI tools (such as Claude, ChatGPT, or Copilot) as a learning partner throughout this assignment. The prompts below are designed to help you understand, debug, and improve your work — not to replace the thinking you need to do yourself.

The right mental model

Think of AI as a senior developer sitting beside you. A good senior developer will explain concepts, point out what is wrong in your code, and suggest improvements. They will not write the entire application for you — because that would mean you learn nothing and cannot maintain the code when something breaks.

Use AI for:

- Understanding what a concept means before you use it.
- Verifying that your planned logic is correct (before coding it).
- Finding and explaining the bug in code you already wrote.
- Getting feedback on finished code to improve quality.

Do not use AI for:

- Generating the full solution or any complete section.
- Producing code you cannot explain line by line.
- Bypassing the thinking required to understand arrays, loops, and form flow.

If you submit AI-generated code, you will likely pass the submission — but fail to understand the material. This will directly hurt your performance in the next assignment and in any technical interview. The loss is yours, not the AI's.

The self-check rule

After completing each part, close your editor and explain out loud what your code does — every line. If you cannot do this for any section, that section is not finished yet. Go back, ask AI to quiz you on it, and close the gap.

Step-by-step prompt guide

The following steps mirror the natural order of building this assignment. Use the prompts at each stage. Replace bracketed placeholders with your actual content.

Step 1 Decompose the assignment before touching code

This assignment has 5 parts that depend on each other. Before writing a single line of PHP, map out all dependencies. Use AI to help you understand the architecture.

Prompts to use:

```
I have a PHP assignment that builds a form-based book manager. The parts are: data array, POST handling, validation, Bootstrap display, and GitHub. In what order should I build these? What does each part depend on?
```

```
Explain to me what PHP sessions are and why I would need session_start() at the very top of the file before any HTML output.
```

```
What is the difference between $_GET, $_POST, and $_SERVER["REQUEST_METHOD"] in PHP? When would I use each?
```

Step 2 Understanding PHP arrays in this context

The core data structure is a multi-dimensional associative array. You need to understand how to read from it, add to it, and loop through it before working on the form.

Prompts to use:

```
Show me how a multi-dimensional associative array works in PHP with a book example. Include at least three entries with keys: id, title, author, genre, year, pages. Do not show me how to build a form yet.
```

```
How do I find the maximum value of a specific key (like "id") across all elements of a PHP array using a loop? I cannot use max() on the array directly.
```

How does `in_array()` work in PHP? Show me how to check if a user-submitted value exists inside a predefined list of allowed values.

Step 3 Planning your validation logic

Write out all your validation rules in pseudocode before touching PHP. AI can verify your logic without writing the code for you.

Prompts to use:

Here are my planned validation rules. Is any rule missing or logically wrong?

- title: required, 3-120 chars
- author: required, must contain a space
- genre: required, must be in a predefined list
- year: required, 4 digits, between 1000 and current year
- pages: required, must be a positive integer

What is the PHP way to check that a string contains at least two words? I want to validate that an author's name has a first and last name. Show me only the condition, not a full script.

How do I get the current year as an integer in PHP to use it as the upper bound for year validation?

Step 4 Debugging form and session issues

The most common bugs in this assignment are: headers already sent, missing `htmlspecialchars`, and re-submission on refresh. Use AI to diagnose these specifically.

Prompts to use:

I am getting the PHP error "headers already sent". My code has `session_start()` at line 3 but I put a space before `<?php`. Why does this cause the error and how do I fix it?

My form does not re-populate after a failed validation. I store submitted data in `$submittedData = $_POST`. Here is my input HTML: [paste your input tag]. What am I missing?

When I refresh the page after adding a book, it re-submits the form and adds a duplicate. How do I fix this? What is the Post/Redirect/Get pattern?

My genre <select> is not keeping the selected value after a failed submission. Here is my loop: [paste your loop]. What is wrong with the selected attribute logic?

Step 5 Bootstrap integration and display logic

Bootstrap errors are usually about using the wrong class combination or missing the is-invalid trigger. Be specific when asking AI.

Prompts to use:

In Bootstrap 5, how do I make a text input show a red border and display a message below it when there is a validation error? Show me just the HTML structure, no PHP.

How do I add a dismissible close button to a Bootstrap 5 alert? What JavaScript attribute do I need and which JS file must be included?

I want to create a Bootstrap table that is striped and hoverable. What classes do I need on the <table> tag? Show me the structure with a header row and one data row.

Explain how Bootstrap's grid system (.container, .row, .col-md-4, .col-md-8) works. I want to put a form on the left taking 4 columns and a table on the right taking 8 columns.

Step 6 GitHub and version control

Many students skip GitHub until the last moment. Use AI to guide you through the git workflow step by step.

Prompts to use:

Walk me through the exact terminal commands to: create a local git repository, add index.php, make my first commit, and push it to an existing empty GitHub repository. Give me the commands in order.

What should I write in my first git commit message for a PHP book manager application? What makes a good commit message?

I get the error "fatal: remote origin already exists" when running git remote add origin. What does this mean and how do I fix it?

Remember: AI is most powerful when you come to it with a specific, well-formed question. Vague inputs produce vague outputs. The clearer your question, the sharper the answer — and the more you will learn from the exchange.

Quick Specification Summary

Field	Validation Rule
title	Required · 3–120 characters
author	Required · Must contain at least two words
genre	Required · Must exist in \$genres array · Validate with in_array()
year	Required · 4-digit integer · between 1000 and current year
pages	Required · Positive integer greater than 0

Technical Requirement	Detail
Framework	Bootstrap 5 via CDN only
File name	index.php (single file)
Session	session_start() before DOCTYPE
Redirect after success	header("Location: index.php") + exit
ID generation	max existing id + 1 (loop, not max() on raw array)
Security	htmlspecialchars() on all echoed user data
GitHub repo name	php-book-library (public)

Good luck 🙌